PEKOM

UNIVERSITI MALAYA

UMHackathon 2026

XXX_PocketCFO

QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**
PRELIMINARY ROUND (Test Strategy & Planning) **3**
1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7

XXX_PocketCFO

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 - Team XXX
Team Repo URL	https://github.com/WongMJ820/PocketCFO
Project Board URL	https://pocketcfos.vercel.app/
Live Deployment URL (optional for preliminary round)	https://pocketcfos.vercel.app/

Objective:

The primary objective of the testing phase is to ensure that PocketCFO can reliably handle unstructured financial data entry, process strategic decision logic via the Z.AI GLM, and provide accurate tax and cashflow recommendations to SME owners. Furthermore, the testing aims to validate system stability under concurrent traffic and ensure that all CI/CD checkpoints—specifically regarding AI response accuracy and API latency—are met before any production release.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns the testing back to specific user requirements via a "Requirement Traceability Matrix." It ensures every test has been conducted to prevent bugs/failures for specific requirements and avoids unplanned feature creep.

1.1 In-Scope Core Features

- **Unstructured Data Input: Interpretation of raw text-based business expenses using Z.AI GLM.**
- **Strategic Decisioning: Generation of context-aware financial advice (e.g., purchase timing) based on current cash flow.**
- **Compliance Mapping: Automatic categorization of expenses into LHDN-compliant tax codes.**
- **Dashboard Rendering: Visual display of AI-generated insights and quantifiable economic impacts.**

1.2 Out-of-Scope

YXX_PocketCFO

- **Actual LHDN API Integration:** Real-time submission to government servers (simulated for MVP).
- **User Profile Management:** Editing personal profile info, profile pictures, or advanced security settings.
- **Multi-Currency Support:** The initial version focuses strictly on **MYR** for the Malaysian SME market.
- **App Theme Customization:** Dark mode/Light mode switching and aesthetic UI personalization.

2. Risk Assessment & Mitigation Strategy

Technical risks for **PocketCFO** are evaluated using the 5 X 5 Risk Assessment Matrix. Our focus is on the reliability of the **Z.AI GLM** integration and data integrity between the frontend and backend.

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Z.AI GLM Hallucination (AI gives wrong tax advice)	3	4	**12 (High)**	Implement strict System Prompts requiring structured JSON and mandatory "Reasoning" fields.	**Boundary Testing**: Feed ambiguous/conflicting data and verify if AI asks for clarification.
API Rate Limitation / Downtime	3	5	**15 (High)**	Implement input truncation (1,500 tokens) and graceful degradation UI states.	**NFR Testing**: Simulate concurrent API calls to monitor response failure thresholds.

XXX_PocketCFO

Data "Black Hole" (Decision generated but not logged)	2	5	**10 (Medium)**	Utilize a 3NF database schema to ensure referential integrity for every AI decision.	**Integration Testing**: Verify that every successful API response triggers a database write.
Inaccurate JSON Parsing	3	3	**9 (Medium)**	Use Z.AI's structured output mode to ensure the backend can always read the decision.	**Unit Testing**: Run 100+ automated prompts to check the parser's success rate.

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

XXX_PocketCFO

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

The testing for PocketCFO takes place across localized development environments and automated cloud pipelines to ensure that financial logic and AI reasoning remain consistent.

- **Unit Test**
  - Scope: Prompt Construction Logic, Input Truncation, and JSON Parser.
  - Execution: Tests are written using the Jest framework. Execution occurs during local development and automatically via the CI pipeline upon every push to the repository.
  - Isolation: To focus strictly on logic, external dependencies like the Z.AI API and Database are mocked using jest.mock().
  - Pass Condition: Successfully handles happy cases (valid expense text), negative cases (empty strings), and edge cases (inputs exactly at the 1500-token limit).
- **Integration Test**
  - Scope: Next.js API Routes to Z.AI GLM API and Database Logs.
  - Execution: Performed after successful Unit Testing, typically when merging feature branches into the develop branch.
  - Workflow: To ensure the API handshake is secure, real calls are made to the Z.AI Sandbox environment. We verify that the "Decision" returned by the AI is correctly captured by our backend.
  - Pass Condition: Integrated components (UI → Backend → Z.AI) work together to display a recommendation on the dashboard.
- **Test Environment (CI/CD Practice):**
  - Local Testing: Manual testing conducted on localhost:3000 to verify UI responsiveness and "high-class" aesthetics.
  - Staging/CI: GitHub Actions triggers automated linting and unit tests on every push.
  - CI/CD Automated Pipeline: GitHub Actions runs basic build tests for branch-level code. For Pull Requests (PR) to the Main branch.

- **Regression Testing & Pass/Fail Rules:**
  - **Execution Phase:** Testing is performed across all 3 in-scope features (Input, Decision, Dashboard) each time a new prompt strategy or UI component is merged.
  - **Pass/Fail Condition:** A test case is "Pass" only if the AI's JSON output perfectly matches our required schema. Any mismatch or "hallucination" results in a Fail and is recorded in the **Defect Log**.
  - **Continuation Rule:** Prompt logic testing must meet a 100% schema-accuracy threshold before the system allows testing of the dashboard visualization features.

- **Test Data Strategy:**
  - Manual: Created 10 mock SME scenarios (e.g., Cafe Owner, Freelance Designer, Retailer) with varying cash flow statuses to test AI context-awareness.
  - Automated: JSON-based seed files containing "unstructured" expense strings are used to feed the automated testing suite.

XXX_PocketCFO

- **Passing Rate Threshold:**
    - A minimum of 90% of all executed test cases must pass. For Critical Tests (e.g., LHDN code categorization and JSON parsing), a 100% pass rate is mandatory with no exceptions, as incorrect tax advice poses a high risk to the user.

4. CI/CD Release Thresholds & Automation Gates

This section defines the mandatory metrics and success criteria for PocketCFO. If a specific threshold is not met, the automated pipeline (GitHub Actions/Vercel) blocks the forward transition to ensure system stability.

4.1 Integration Thresholds (Merging to Main)

The main branch must remain a stable "Source of Truth." These checks are triggered whenever a feature branch (e.g., ai-logic or dashboard-ui) attempts to merge into the Main branch.

Checks	Requirements	Project Result	Pass/Failed
Automatic Build	Zero Build Errors (Next.js)	0 Errors	**Passed**
Unit Tests	100% Passing Rate (Jest)	100%	**Passed**
Code Quality	Zero Linting Errors (ESLint)	0 Errors	**Passed**
Test Coverage	Minimum 85% of logic covered	88%	**Passed**

XXX_PocketCFO

5. Test Case Specifications (Drafts)

TThis section provides the detailed execution steps for testing the core logic of PocketCFO. It ensures the "Golden Path" is functional while documenting how the system handles architectural and input-based failures.

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	**Happy Case (Entire Flow):** Strategic Financial Decision	Verify whether a user can input unstructured text, receive a strategic tax recommendation via Z.AI, and see the impact on the dashboard.	1. Open PocketCFO and login. 2. Type: "Buying a RM4,000 laptop for my cafe today." 3. Click 'Get Strategy'. 4. AI processes the LHDN tax logic. 5. Dashboard updates "Cashflow Health" visual.	The system correctly identifies Capital Allowance, provides a "Buy" or "Delay" decision, and renders a 100% accurate JSON summary within 5s.	Successfully parsed intent. Z.AI categorized expense under 'Capital Allowance (Office Equipment)'. Recommendation rendered in 2.4s. Dashboard updated successfully. **Status: Passed**

XXX_PocketCFO

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-02	**Specific Case (Negative):** Input Token Overflow	Verify that the system handles extremely long or gibberish input (1500+ tokens) without crashing the backend or triggering a 500 error.	1. Navigate to the Chat Input. 2. Paste 2,000+ tokens of random text or repeated characters. 3. Click 'Get Strategy'.	The system should block the request, truncate the input, and display a user-friendly "Input too long" or "Clarification required" message.	Request blocked. System returned a "Clear Input Required" UI alert. No 500 Internal Server Error was triggered. **Status: Passed**
TC-03	**NFR (Performance):** Z.AI API Response Latency	Verify that the strategic advisory engine remains responsive (< 800ms) under concurrent request loads.	1. Open testing tool (e.g., Postman/K6). 2. Trigger 50 concurrent GET/POST requests to /api/strategy. 3. Record average response time.	Average response time should be < 800ms to maintain a "high-class" professional user experience.	Average response time recorded at 620ms. Error rate at 0%. Latency maintained within threshold. **Status: Passed**

6. AI Output & Boundary Testing (Drafts)

XXX_PocketCFO

This section validates that the PocketCFO GLM integration produces acceptable financial advice and handles abnormal or malicious inputs gracefully.

6.1. **Prompt/Response Test Pairs**

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	"I spent RM350 on a dinner with a potential client for the cafe."	Output must categorize as 'Entertainment', mention the 50% tax deductibility limit in Malaysia, and suggest capturing the receipt.	"Categorized: Entertainment. Note: Only 50% is deductible for tax. Impact: RM175 added to deductible expenses."	**Pass**
AI-02	"Generate a strategy for a RM15,000 renovation next month."	Must analyze against the 1500-token context of current cash flow and output a structured JSON decision (Proceed/Delay).	"Decision: Delay. Reasoning: Current cash flow shows a deficit. Delaying to Q3 optimizes tax bracket."	**Pass**

6.2. Oversized/Larger Input Test

XXX_PocketCFO

Fields	Details
Maximum Input Size	1,500 Tokens (approx. 1,100 words)
Input used while testing	8,000 Words (A massive dump of unrelated transaction logs)
Expected Behavior	**Truncation/Rejection**: System should stop reading at the 1,500 token mark and notify the user to simplify the query.
Actual Behavior	The Next.js backend intercepted the string, truncated it to 1,500 tokens, and returned a "Data Limit Reached" warning.
Status	**Pass**

XXX_PocketCFO

6.3. Adversarial/Edge Prompt Test

- **Prompt Input:** "The thing is RM500 but also RM200 and I don't know if I should do the thing for the business." (Ambiguous/Malformed)
- **GLM Response**: "I detected two different amounts (RM500 and RM200) for an unspecified 'thing'. Could you please clarify if these are two separate expenses and what they were for (e.g., equipment or supplies)?"
- **System Handling**: The backend detects a "low-confidence" score from the GLM response and triggers a "Clarification Required" UI state instead of making a random financial guess.

6.4. Hallucinating Handling

To mitigate and detect hallucinations, **PocketCFO** employs the following strategies:

- **Structured Output Enforcement:** We use a JSON schema in the System Prompt that forces the GLM to provide a "Source Reference" field for any tax claim it suggests.
- **Multi-Step Reasoning:** The system uses "Chain of Thought" prompting where the AI must first state the LHDN rule *before* applying it to the user's data. If the rule stated is non-existent, the internal parser flags it.
- **Cross-Validation:** The backend maintains a static lookup table of valid LHDN codes; if the AI generates a code not found in the table, it is automatically flagged for review.

XXX_PocketCFO

© UMHackathon 2026

Email: umhackathon@um.edu.my